

BCC202 - Estruturas de Dados I

Aula 02: Princípios da Programação em C

Pedro Silva

Universidade Federal de Ouro Preto – UFOP

Departamento de Computação – DECOM

Email: silvap@ufop.edu.br



Conteúdo

Definição de um Problema

Resolvendo o problema

Uso de memória

Modularização

Compilação

Conclusão

Exercícios

Bibliografia

Definição de um Problema

Um problema de exemplo

Desejo calcular a média das notas dos 50 alunos de uma turma de Estruturas de Dados I do semestre corrente. São três provas, todas com o mesmo peso (média aritmética) e todas avaliadas entre zero e dez (valores fora do intervalo são erros). O nosso objetivo é criar um programa para realizar os cálculos para ajudar o professor.

Nome do aluno	Nota 1	Nota 2	Nota 3	Média
Aluno 1	6.0	7.0	8.0	7.0
Aluno 2	8.0	7.0	6.0	7.0
Aluno 3	10.0	10.0	10.0	10.0
⋮	⋮	⋮	⋮	⋮
Aluno 50	3.0	1.0	5.0	3.0

Resolvendo o problema

Por onde começamos?

Por onde começamos?

Entendendo a linguagem que utilizaremos: C.

Construções Fundamentais da linguagem C

A linguagem C provê as **construções fundamentais** de fluxo de controle necessárias para programas bem estruturados:

- ▶ agrupamentos de comandos;
- ▶ **estruturas de decisão** - tomadas de decisão (*if-else*);
- ▶ laços com testes de **encerramento no início** (*while, for*);
- ▶ laços com testes de **encerramento no fim** (*do-while*);
- ▶ **seleção** de um dentre um conjunto de possíveis casos (*switch*);
- ▶ acesso a **apontadores** e a habilidade de fazer aritmética com endereços.

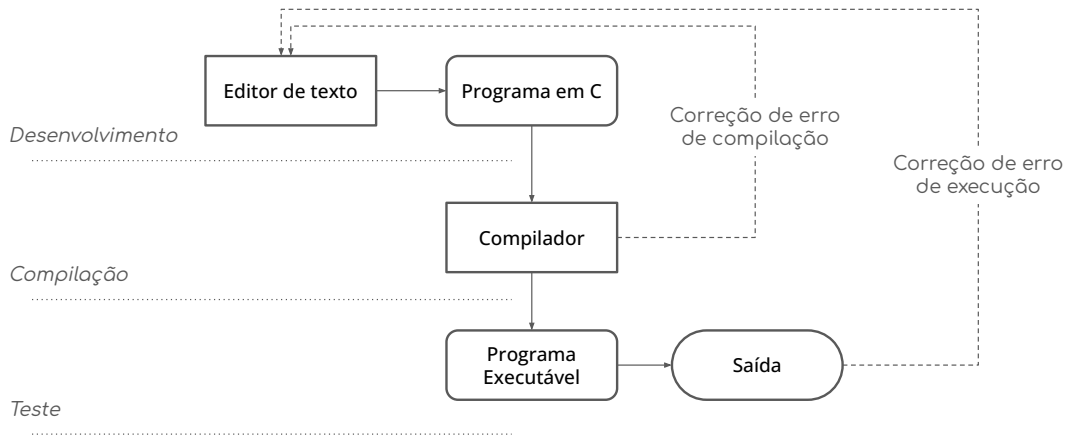
Limitações da linguagem C

A linguagem C não provê operações para:

- ▶ **manipulação direta** de objetos compostos, tais como cadeias de caracteres;
- ▶ nem facilidades de **entrada e saída**: não há comandos *READ* e *WRITE*.

Os mecanismos são fornecidos por
funções explicitamente chamadas (importadas)

Ciclo de Desenvolvimento de um programa em C



Execução de um Programa em C

Mas o que é o processo de compilação?

Execução de um Programa em C

Mas o que é o processo de compilação?

Para executar um programa em C, devemos antes converter o programa em C para um programa em linguagem de máquina (**compilação**).

Por onde começamos?

- ✓ Entendendo a linguagem que utilizaremos: C.
Armazenando os dados. Mas onde?

Variáveis

- ▶ Variável representa um **espaço na memória** para armazenar determinado tipo de dados.
- ▶ Na linguagem C, todas as variáveis devem ser **explicitamente** declaradas:
`<tipo_variavel> nome_variavel;`

```
int main() {  
    int a = 5;  
    char c = 'p';  
    // ... mais código  
}
```

Endereço	Valor
...	...
0x0100	0000 0000
0x0101	0000 0000
0x0102	0000 0000
0x0103	0000 0101
0x0104	0111 0000
...	...

Diagram illustrating memory layout for variable `c` (char):

- Addresses 0x0100 to 0x0104 are grouped by a bracket labeled `5`, indicating the memory span for the character.
- Addresses 0x0103 and 0x0104 are grouped by a bracket labeled `'p'`, indicating the stored value.

Por onde começamos?

- ✓ Entendendo a linguagem que utilizaremos: C.
- ✓ Armazenando os dados. Mas onde?

Como interagimos com o usuário (I/O)?

Impressão e Leitura de Dados:

Impressão no Terminal

A linguagem C, tem a função `printf(...)`, que como o próprio nome diz, imprime.

Lendo do Teclado:

A função `scanf(...)` permite ler valores do teclado.

Tipo de Dado

- ▶ A função `scanf` recebe dois parâmetros: o **tipo** do dado que ela vai ler, e um **endereço de qual variável** onde ela deverá colocar o valor lido.
- ▶ A função `printf` usualmente também precisa saber o **tipo** de dado que será impresso e a **variável**.

Especificador de Tipos

Tipo	Especificação
%c	especifica um <i>char</i>
%d ou %i	especifica um <i>int</i>
%ld ou %li	especifica um <i>long int</i>
%lld ou %lli	especifica um <i>long long int</i>
%f	especifica um <i>double</i> (ou <i>float</i>)
%e	especifica um <i>double</i> (ou <i>float</i>) no formato científico
%g	especifica um <i>double</i> (ou <i>float</i>) no formato mais apropriado (%f ou %e)
%s	especifica uma cadeia de caracteres
%u	especifica um <i>int</i> sem sinal (<i>unsigned int</i>)
%lu	especifica um <i>long unsigned int</i>
%llu	especifica um <i>long long unsigned int</i>

Exemplo do uso dos tipos

```
// stdio: uma biblioteca de standard I/O, ou seja,  
//      entrada e saída padrão  
#include <stdio.h>  
  
int main() {  
    ....  
    int idade;  
    printf("Qual sua idade?\n");  
    scanf("%d", &idade);  
    printf("Idade informada: %d", idade);  
    ....  
    return 0;  
    // A função main devolve um inteiro para informar ao  
    // sistema operacional sobre o fim da execução do prog  
}
```

Por onde começamos?

- ✓ Entendendo a linguagem que utilizaremos: C.
- ✓ Armazenando os dados. Mas onde?
- ✓ Como interagimos com o usuário (I/O)?

Como vamos calcular a média?

Principais Tipos de Operadores

- ▶ A Linguagem C oferece uma gama variada de operadores, entre **unários** e **binários**.
 - ▶ Aritméticos.
 - ▶ Atribuição.
 - ▶ Relacionais.
 - ▶ Lógicos.
- ▶ Em C, um operador binário é escrito/opera sobre **dois** operandos e operador unário **precede** seu **único** operando.

Operadores Aritméticos

Os operadores **binários** aritméticos são:

adição (+)

subtração (−)

multiplicação (*)

divisão (/)

Uma atribuição é uma expressão cujo valor resultante corresponde ao valor atribuído.

```
int x, y;  
x = y = 5;  
// a ordem de avaliação é da direita para esquerda
```

Operadores de Atribuição Compostos

A variável à esquerda do sinal da atribuição também aparece à direita. Exemplo:

```
int x = 10;  
x = x + 1;
```

O que equivale a:

```
int x = 10;  
x += 1;
```

De forma análoga, existem: $- =$; $* =$; $/ =$

Operadores de Incremento e Decremento

Operadores não convencionais que possuem precedência ao operador unário e são utilizados para incrementar/decrementar **uma unidade** nos valores armazenados em variáveis. Exemplo:

```
// incrementa de uma unidade o valor de n
x = n++;
// análogo para o decremento em n--
```

Os incrementos podem ser **pré-fixados** (antes da variável - “++n”) ou **pós-fixado** (após a variável - “n++”):

```
int n = 5;
x = n++; // atribui 5 a x.
n = 5;
x = ++n; //atribui 6 a x.
```

Por onde começamos?

- ✓ Entendendo a linguagem que utilizaremos: C.
- ✓ Armazenando os dados. Mas onde?
- ✓ Como interagimos com o usuário (I/O)?
- ✓ Como vamos calcular a média?

Como checamos se as notas estão em um intervalo válido?

Condicional (*if*)

Avalia uma expressão *booleana* e redireciona o fluxo de execução baseado no resultado avaliado (verdadeiro ou falso).

```
if(expressao_booleana) {  
    bloco_comandos_1;  
}  
bloco_comandos_2;
```

Condicional (*if/else*)

Outra possível construção com o comando *if* acrescenta o bloco *else*, a ser executado apenas se a expressão *booleana* resultar em falso.

```
if(expressao_booleana) {  
    bloco_comandos_1;  
} else {  
    bloco_comandos_2;  
}  
bloco_comandos_3;
```

Operadores Relacionais e Lógicos

Relacionais

São usados para comparar dois valores (e.g., $==$; $!=$; $<$, $\leq$)

Lógicos

São usados para combinar expressões *booleanas*.

- ▶ $\&\&$ operador binário AND.
- ▶ $\|\|$ operador binário OR.
- ▶ $!$ operador unário de negação.

Seleção (*switch*)

Comando para selecionar um dentre um conjunto de possíveis casos.

```
switch (expr) {  
    case op1:  
        // comandos executados para expr == op1  
        ...  
        break;  
    case op2:  
        // comandos executados para expr == op2  
        ...  
        break; // é obrigatório?  
    default:  
        // executado se "expr" for diferente de todos casos  
        ...  
        break;  
}
```

Por onde começamos?

- ✓ Entendendo a linguagem que utilizaremos: C.
 - ✓ Armazenando os dados. Mas onde?
 - ✓ Como interagimos com o usuário (I/O)?
 - ✓ Como vamos calcular a média?
 - ✓ Como checamos se as notas estão em um intervalo válido?
- Como vamos iterar por cada um dos alunos?

Estrutura de repetição (*while*)

Repetições iterativas são programadas mediante a construção de laços (ou ciclos).

O comando *while*

Repete a execução de um bloco de comandos **enquanto** determinada condição for satisfeita. A sintaxe deste comando é:

```
while(expressao_booleana) {  
    bloco_comandos_1;  
}  
bloco_comandos_2;
```

O comando *while* e *loops* infinitos

Loop “infinito” pode ser útil?

O comando *while* e *loops* infinitos

Loop “infinito” pode ser útil?

```
while (true) {  
    ...  
    if (expressao_booleana) {  
        break;  
    }  
}
```

Estruturas de repetição (*for*)

Equivalente ao comando *while*, porém com uma sintaxe mais concisa e compacta.

```
for (expressao_inicializacao;  
    expressao_booleana;  
    expressao_atualizacao) {  
    bloco_comandos_1;  
}  
bloco_comandos_2;
```

Exemplo de um *for* com 100 iterações.

```
for (int x = 0; x < 100; x++) {  
    printf("%d \n", x);  
}
```

A “*expressao_inicializacao*” é executada uma única vez e usualmente **declara** e inicializa uma variável, a ser incrementada ou decrementada nas iterações do laço.

Estruturas de repetição (*do-while*)

Ao contrário dos comandos *for* e *while*, a expressão *booleana* é avaliada no *final* do laço.

```
do {  
    bloco_comandos_1;  
} while(expressao_booleana);  
bloco_comandos_2;
```

A construção dos laços com testes no final é especialmente útil quando *precisamos executar um passo da iteração* antes de avaliar a condição de repetição.

Estruturas de interrupção de laços de repetição

break

- ▶ Quando utilizado dentro de *loop*, **interrompe e termina** a execução deste.
- ▶ A execução prossegue com os comandos subsequentes ao bloco.

continue

- ▶ **Interrompe** a execução dos comandos de um **laço**.
- ▶ O laço não é automaticamente finalizado.
- ▶ A execução de uma interação do *loop* é interrompida, passando para a próxima interação.

Por onde começamos?

- ✓ Entendendo a linguagem que utilizaremos: C.
- ✓ Armazenando os dados. Mas onde?
- ✓ Como interagimos com o usuário (I/O)?
- ✓ Como vamos calcular a média?
- ✓ Como checamos se as notas estão em um intervalo válido?
- ✓ Como vamos iterar por cada um dos alunos?

Não há uma forma melhor de estruturar o nosso código?

Modularização de código

Em programas estruturados, é recomendado:

- ▶ **dividir** as grandes tarefas de computação em tarefas menores;
- ▶ utilizar os **resultados parciais** das tarefas menores para compor o resultado final.

Bons programas em geral são compostos por diversas funções, sendo que **cada função** implementa **uma funcionalidade**.

```
tipo_do_retorno nome_da_funcao(lista_de_parametros) {  
    // bloco de comandos, incluindo return, se cabível  
    corpo_da_funcao;  
}
```

Conceitos importantes: **interface/especificação**, **implementação** e **escopo de variáveis**.

Uso de memória

Uso de Memória por um Programa

Informalmente, podemos dizer que existem três maneiras de reservar espaço de memória para o armazenamento de informações.

- ▶ Uso de variáveis globais e estáticas o espaço existe enquanto o programa estiver sendo executado.
- ▶ Variáveis locais (Pilha) - o espaço existe enquanto a função que declarou a variável estiver sendo executado, sendo liberado para outros usos quando a execução da função termina.
- ▶ Requisitando ao sistema em tempo de execução um espaço de memória de determinado tamanho - o espaço alocado dinamicamente permanece reservado até que seja explicitamente liberado,

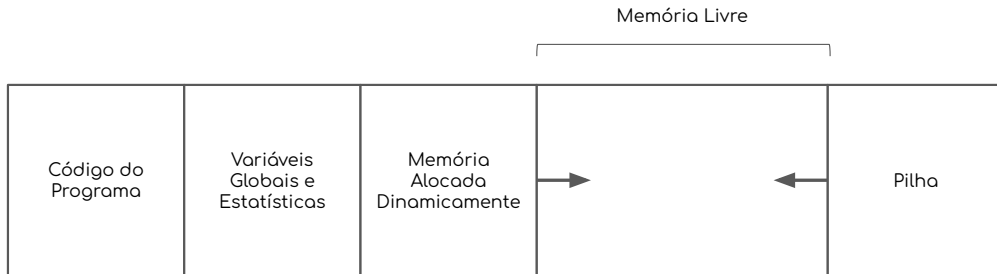
Divisão da memória

Em geral, a memória utilizada por um programa de computador é **dividida** em:

- ▶ Segmento de Código
- ▶ Segmento de Dados
- ▶ *Heap*
- ▶ Pilha (*stack*)

Ilustração: Alocação de Memória

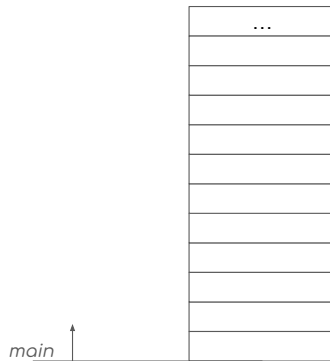
A seguir, um esquema didático que ilustra a distribuição de memória, feita pelo sistema operacional, para um programa qualquer:



Pilha de Variáveis Durante a Execução de um Programa

Exemplo de alocação de variáveis na pilha durante a execução de um programa.

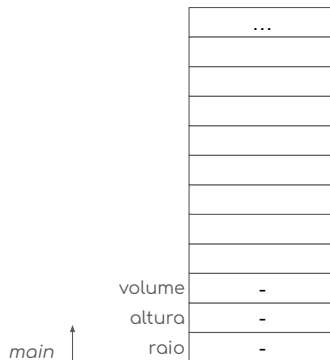
```
int calcula_volume(int h, int r) {  
    float v;  
    v = PI * r * r * h;  
    return v;  
}  
  
int main() {  
    float raio, altura, volume;  
    scanf("%f %f", &raio, &altura);  
    volume = calcula(raio, altura);  
    printf("Volume: %.2f\n", volume);  
    return 0;  
}
```



Pilha de Variáveis Durante a Execução de um Programa

Exemplo de alocação de variáveis na pilha durante a execução de um programa.

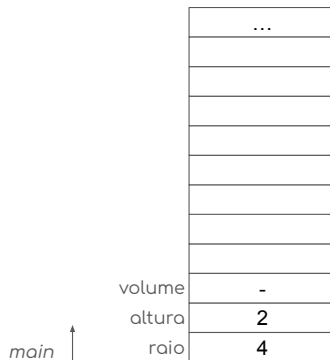
```
int calcula_volume(int h, int r) {  
    float v;  
    v = PI * r * r * h;  
    return v;  
}  
  
int main() {  
    float raio, altura, volume;  
    scanf("%f %f", &raio, &altura);  
    volume = calcula(raio, altura);  
    printf("Volume: %.2f\n", volume);  
    return 0;  
}
```



Pilha de Variáveis Durante a Execução de um Programa

Exemplo de alocação de variáveis na pilha durante a execução de um programa.

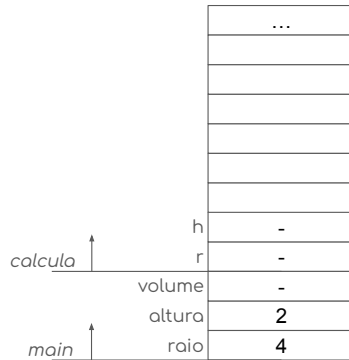
```
int calcula_volume(int h, int r) {  
    float v;  
    v = PI * r * r * h;  
    return v;  
}  
  
int main() {  
    float raio, altura, volume;  
    scanf("%f %f", &raio, &altura);  
    volume = calcula(raio, altura);  
    printf("Volume: %.2f\n", volume);  
    return 0;  
}
```



Pilha de Variáveis Durante a Execução de um Programa

Exemplo de alocação de variáveis na pilha durante a execução de um programa.

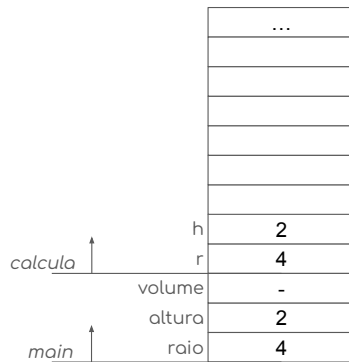
```
int calcula(int r, int h) {  
    float v;  
    v = PI * r * r * h;  
    return v;  
}  
  
int main() {  
    float raio, altura, volume;  
    scanf("%f %f", &raio, &altura);  
    volume = calcula(raio, altura);  
    printf("Volume: %.2f\n", volume);  
    return 0;  
}
```



Pilha de Variáveis Durante a Execução de um Programa

Exemplo de alocação de variáveis na pilha durante a execução de um programa.

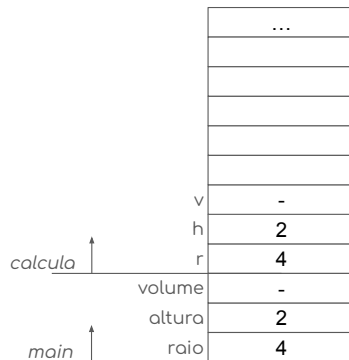
```
int calcula(int r, int h) {  
    float v;  
    v = PI * r * r * h;  
    return v;  
}  
  
int main() {  
    float raio, altura, volume;  
    scanf("%f %f", &raio, &altura);  
    volume = calcula(raio, altura);  
    printf("Volume: %.2f\n", volume);  
    return 0;  
}
```



Pilha de Variáveis Durante a Execução de um Programa

Exemplo de alocação de variáveis na pilha durante a execução de um programa.

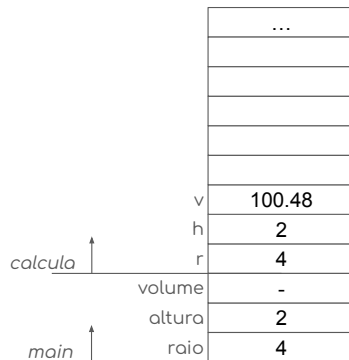
```
int calcula(int r, int h) {  
    float v;  
    v = PI * r * r * h;  
    return v;  
}  
  
int main() {  
    float raio, altura, volume;  
    scanf("%f %f", &raio, &altura);  
    volume = calcula(raio, altura);  
    printf("Volume: %.2f\n", volume);  
    return 0;  
}
```



Pilha de Variáveis Durante a Execução de um Programa

Exemplo de alocação de variáveis na pilha durante a execução de um programa.

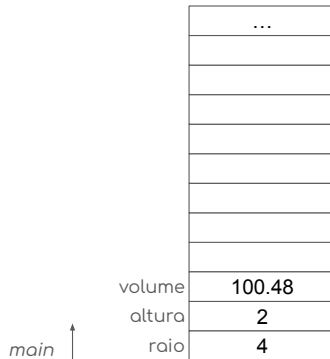
```
int calcula(int r, int h) {  
    float v;  
    v = PI * r * r * h;  
    return v;  
}  
  
int main() {  
    float raio, altura, volume;  
    scanf("%f %f", &raio, &altura);  
    volume = calcula(raio, altura);  
    printf("Volume: %.2f\n", volume);  
    return 0;  
}
```



Pilha de Variáveis Durante a Execução de um Programa

Exemplo de alocação de variáveis na pilha durante a execução de um programa.

```
int calcula(int r, int h) {  
    float v;  
    v = PI * r * r * h;  
    return v;  
}  
  
int main() {  
    float raio, altura, volume;  
    scanf("%f %f", &raio, &altura);  
    volume = calcula(raio, altura);  
    printf("Volume: %.2f\n", volume);  
    return 0;  
}
```



Podem ser utilizadas para fazer a comunicação entre funções. Sobre variáveis globais:

- ▶ Declaradas **fora** do corpo das funções.
- ▶ Não são armazenadas na pilha de execução.
- ▶ Existem enquanto o programa estiver sendo **executado**.

Considerações...

Não são armazenadas na pilha.

- ▶ Visível **somente** dentro do arquivo que a declara.
- ▶ Variáveis estáticas dentro de funções:
 - ▶ A variável também é armazenada numa área de **memória estática** que existe enquanto o programa está sendo executado.
 - ▶ Continuam existindo **antes e depois** de a função ser executada.
 - ▶ Existem enquanto o programa estiver sendo **executado**.
 - ▶ São **visíveis** dentro da função que as declarou.

Variável Estática dentro de Função

```
void imprime ( float a ) {  
    static int n = 1;  
    printf(" %f ", a);  
    if ((n % 5) == 0)  
        printf(" \n ");  
    n++;  
}
```

O que será impresso?

Ponteiro de Variáveis

- ▶ A linguagem C permite o armazenamento e a manipulação de valores de endereços de memória.
- ▶ Para cada tipo existente, há um tipo ponteiro que pode armazenar endereços de memória onde existem valores do tipo correspondente armazenado.
- ▶ Uma variável do tipo ponteiro tem a seguinte sintaxe geral:

```
...  
<tipo_variavel>* nome_variavel;  
...
```

Operadores Para Atribuir e Acessar Endereços de Memória

- ▶ O operador unário `&`, **endereço de**, aplicado a variáveis, resulta no endereço da posição da memória reservada para a variável.
- ▶ O operador unário `*`, **conteúdo de**, aplicado a **variáveis do tipo ponteiro**, acessa o conteúdo do endereço de memória armazenado pela variável ponteiro.

Exemplo

- Qual valor será impresso ao executar o código a seguir?

```
int main () {  
    int a, b, *p;  
    a = 2;  
    *p = 3;  
    b = a + (*p);  
    printf(" %d ", b);  
    return 0;  
}
```

Exemplo de Código

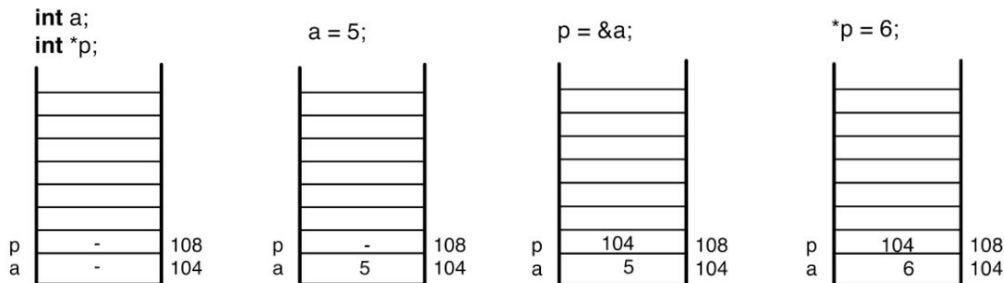
Considere o trecho de código mostrado na figura abaixo.

```
...  
int a; //variável do tipo inteiro  
int* p; // variável do tipo ponteiro para inteiro  
p = &a; //p recebe o endereço de a  
*p = 8; //o conteúdo de p é alterado para 8  
...
```

O que ocorre na pilha de execução?

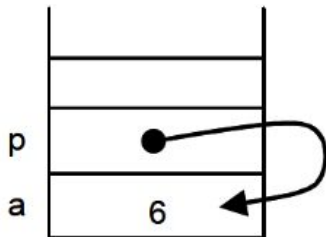
Pilha de Execução: código anterior

Nas figuras a seguir, os números à direita são valores fictícios dos espaços na memória.



Representação Gráfica do Valor de um Ponteiro

Em vez de criarmos valores fictícios para os endereços de memória no nosso esquema ilustrativo da memória, podemos desenhar setas graficamente, sinalizando que um ponteiro aponta para uma determinada variável.



Vantagem

A possibilidade de manipular ponteiros de variáveis é uma das maiores **potencialidades** de C.

Desvantagem

O uso **indevido** desta manipulação é o maior causador de programas que não só não funcionam como, pior ainda, podem gerar efeitos colaterais não previstos.

Passagem de Parâmetros por Referência

Os ponteiros oferecem meios de alterarmos valores de variáveis acessando-as indiretamente.

- ▶ As funções não podem alterar diretamente valores de variáveis da função que fez a chamada.
- ▶ E se passarmos para uma função os valores dos endereços de memória onde suas variáveis estão armazenadas...

Tentativa 1

```
#include <stdio.h>

// funcao troca
void troca (int x, int y ) {
    int temp = x;
    x = y;
    y = temp;
}

int main () {
    int a = 5, b = 7;
    troca(a, b);
    printf("%d %d \n", a, b);
    return 0;
}
```

Quais valores serão impressos?

Tentativa 2

```
#include <stdio.h>

/* funcao troca */
void troca (int *px, int *py ) {
    int temp;
    temp = *px;
    *px = *py;
    *py = temp;
}

int main () {
    int a = 5, b = 7;
    troca(&a, &b);
    printf("%d %d \n", a, b);
    return 0;
}
```

Quais valores serão impressos?

Agora fica explicado por que passamos o endereço das variáveis para a função *scanf*, pois, caso contrário, a função não conseguiria devolver os valores lidos.

Por onde começamos?

- ✓ Entendendo a linguagem que utilizaremos: C.
 - ✓ Armazenando os dados. Mas onde?
 - ✓ Como interagimos com o usuário (I/O)?
 - ✓ Como vamos calcular a média?
 - ✓ Como checamos se as notas estão em um intervalo válido?
 - ✓ Como vamos iterar por cada um dos alunos?
 - ✓ Não há uma forma melhor de estruturar o nosso código?
- E se precisássemos armazenar os dados e não só calcular a média?

Usados para armazenar um conjunto de valores na memória do computador.

- ▶ Os valores são armazenados em **sequência**, um após o outro.
- ▶ É possível acessar **qualquer valor** do conjunto diretamente.
- ▶ Ao declarar um vetor, devemos informar o número **máximo** de elementos que poderá ser **armazenado**.
- ▶ Tipo **homogêneo** de dado: num vetor, só podemos armazenar valores de um mesmo tipo.

Sintaxe para declaração de um vetor

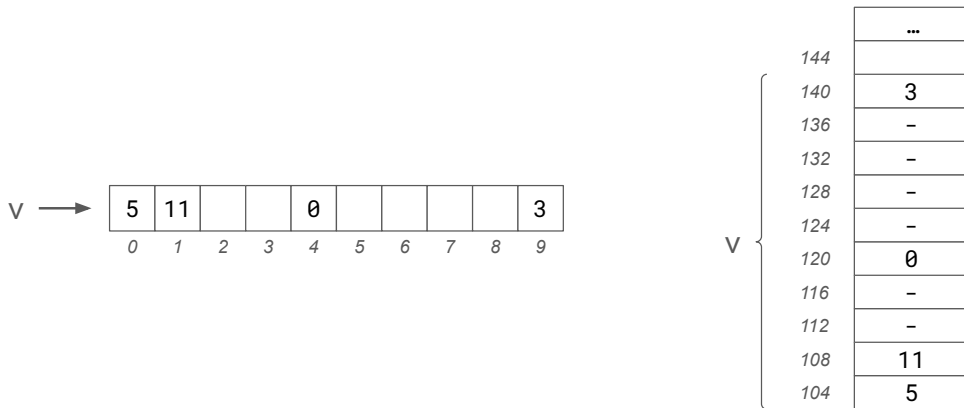
```
...  
<tipo_do_vetor> nome_do_vetor_[tamanho_do_vetor];  
...
```

Exemplo

```
...  
int v[4];  
v[0] = 1;  
v[1] = 2;  
v[2] = 4;  
v[3] = 8;  
/*ou ainda*/  
int valores[] = {3,5,7};  
...
```

Armazenamento do vetor

Ilustração do armazenamento de vetores na memória do computador (pilha no lado direito, representação gráfica no lado esquerdo).



Vetores e Ponteiros

Considerando `int v[10];`: `v` é uma constante com o valor do endereço inicial do vetor.

Relação entre Vetor e Ponteiro

O nome de um vetor é um ponteiro para o tipo do elemento do vetor. No exemplo anterior: `v` é `int*`.

Exemplo

```
...  
int v[10];  
... /*valores são inseridos no vetor*/  
int* u = v;  
u[0] = 4;  
u[1] = v[0] + 2;
```

Quais os valores armazenados em `u[1]` e `v[1]`?

Aritmética de ponteiros para vetores

Em C é possível **somar e subtrair ponteiros**, desde que o valor do ponteiro resultante aponte para **dentro da área reservada** para o vetor.

Com isso, um vetor tem as seguintes equivalências, dado $v[10]$:

$v + 0$	$\rightarrow$	<i>aponta para o primeiro elemento do vetor</i>
$v + 1$	$\rightarrow$	<i>aponta para o segundo elemento do vetor</i>
$v + 2$	$\rightarrow$	<i>aponta para o terceiro elemento do vetor</i>
$\vdots$	$\vdots$	$\vdots$
$v + 9$	$\rightarrow$	<i>aponta para o último elemento do vetor</i>

Equivalências

- ▶ $\&v[i]$ é equivalente a escrever $(v + i)$.
- ▶ $v[i]$ é equivalente a escrever $*(v + i)$

A **forma indexada** é
mais clara e adequada.

Passagem de Vetores para Funções

Passar um vetor para uma função consiste em passar o endereço da primeira posição do vetor.

- ▶ Se passarmos um valor de endereço, a função chamada deve ter um parâmetro do **tipo ponteiro** para armazenar este valor.
- ▶ Assim, se passarmos para uma função um vetor de *int*, devemos ter um parâmetro do tipo *int**, capaz de armazenar endereços de inteiros.
- ▶ **Importante:** A expressão “*passar um vetor para uma função*” deve ser interpretada como “*passar o endereço inicial do vetor*”.

Os elementos do vetor **não são copiados** para a função, o argumento copiado é apenas o **endereço** do primeiro elemento.

Passagem de Vetores para Funções

É passado para a função o endereço do primeiro elemento do vetor (e não os elementos propriamente ditos).

```
#include <stdio.h>

void incr_vetor ( int n, int *v ) {
    for (int i = 0; i < n; i++)
        v[i]++;
}

int main () {
    int a[ ] = {1, 3, 5};
    incr_vetor(3, a);
    printf("%d %d %d \n", a[0], a[1], a[2]);
    return 0;
}
```

Quais valores serão impressos?

Usadas para conjuntos **bidimensionais** de dados.

- ▶ Matrizes devem ser declaradas para que o **espaço apropriado de memória seja reservado**.
- ▶ As duas dimensões da matriz devem ser especificadas: **número de linhas** e **número de colunas**.
- ▶ Tipo **homogêneo** de dados: numa matriz, só podemos alocar valores de um mesmo tipo.

Sintaxe para declaração de uma matriz

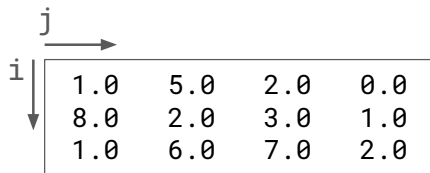
```
...  
<tipo_da_matriz> nome_matriz [num_linhas][num_colunas];  
...
```

Exemplo

```
...  
int v[2][2] = {{0,1},  
               {2,3}};  
/* os valores da matriz poderiam também ser  
 * especificados usando um laço de repetições  
 */  
...
```

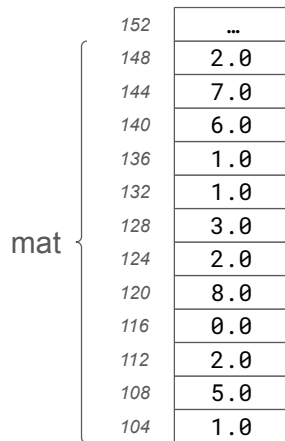
Armazenamento da matriz

A figura a seguir ilustra o armazenamento de matrizes na memória do computador (representação gráfica no lado direito, pilha no lado esquerdo).



A 3x4 matrix is shown with row index *i* and column index *j*. The values are as follows:

	<i>j</i> →			
↓ <i>i</i>	1.0	5.0	2.0	0.0
	8.0	2.0	3.0	1.0
	1.0	6.0	7.0	2.0



The matrix is stored in memory as a single column of values, with addresses on the left. The values are as follows:

152	...
148	2.0
144	7.0
140	6.0
136	1.0
132	1.0
128	3.0
124	2.0
120	8.0
116	0.0
112	2.0
108	5.0
104	1.0

Por onde começamos?

- ✓ Entendendo a linguagem que utilizaremos: C.
 - ✓ Armazenando os dados. Mas onde?
 - ✓ Como interagimos com o usuário (I/O)?
 - ✓ Como vamos calcular a média?
 - ✓ Como checamos se as notas estão em um intervalo válido?
 - ✓ Como vamos iterar por cada um dos alunos?
 - ✓ Não há uma forma melhor de estruturar o nosso código?
 - ✓ E se precisássemos armazenar os dados e não só calcular a média?
- Não há uma forma melhor de armazenar as informações de cada aluno?

Tipos estruturados

Uma estrutura em C permite definir um tipo de dado cujos campos são compostos de vários valores de diversos tipos (**tipo heterogêneo de dado**).

Exemplo de struct

```
#include <stdio.h>

struct ponto {
    float x;
    float y;
};

int main () {
    struct ponto p;
    printf("Digite as coordenadas do ponto(x y): ");
    scanf("%f %f", &p.x, &p.y);
    printf("O ponto fornecido foi: (%.2f,%.2f)\n", p.x, p.y);
    return 0;
}
```

Definição de Novos Tipos

Em C é possível **definir e criar** nomes de tipos para as estruturas. Exemplo:

```
typedef struct ponto Ponto;  
struct ponto {  
    float x;  
    float y;  
};  
  
typedef struct { // forma alternativa  
    int x;  
    int y;  
} Ponto;
```

Assim *Ponto* passa a **representar** a *struct ponto*. Após essas definições, podemos declarar variáveis da seguinte maneira:

```
Ponto p; // não precisa ser struct ponto p
```

Modularização

É um conceito bastante antigo que consiste em **dividir** um programa em **componentes individuais**, chamados módulos, que, uma vez integrados, atendem aos requisitos do problema.

- ▶ Reúso.
- ▶ Legibilidade.
- ▶ Manutenibilidade.
- ▶ Dependabilidade.

Formas de modularização em C e granularidade dos módulos.

Programa para Conversão de Temperatura

Seja um programa que tem a finalidade de converter valores de temperatura dados em Celsius para Fahrenheit.

converte.h

```
// especificação do módulo de conversão
float converte_celsius_fahrenheit (float c);

// especificação de outras funções para conversão
// de temperatura
```

Programa para Conversão de Temperatura

Implementação da especificação.

converte.c

```
// Implementação do módulo de conversão
// inclusão da especificação do módulo
#include "converte.h"

float converte_celsius_fahrenheit (float c) {
    float f;
    f = 1.8 * c + 32;
    return f;
}

// implementação de outras funções para conversão de
// temperatura
```

Módulo Principal

principal.c

```
// inclusão do módulo de conversão de temperatura
#include "converte.h"
#include <stdio.h>

int main () {
    float t1; // espaço para armazenar temperatura em Celsius
    float t2; // espaço para armazenar temperatura em Fahrenheit
    printf("Entre com temperatura em Celsius: ");
    // captura valor entrado via teclado
    scanf("%f",&t1);
    // faz a conversão
    t2 = converte_celsius_fahrenheit(t1);
    // exibe resultado
    printf("Fahrenheit: %f\n", t2);
    return 0;
}
```

Compilação

Estratégia de Compilação

Compilando tudo simultaneamente

```
> gcc -o exe converte.c principal.c
```

Há algum problema nisso?

Compilando tudo simultaneamente

```
> gcc -o exe converte.c principal.c
```

Há algum problema nisso?

- ▶ Se alterarmos a implementação de um determinado módulo precisaremos re-compilar os outros.
- ▶ ALTERNATIVA: Compilar os módulos separadamente e depois ligar os diversos módulos-objetos gerados para criar um executável.

Compilando tudo simultaneamente

```
> gcc -o exe converte.c principal.c
```

Há algum problema nisso?

- ▶ Se alterarmos a implementação de um determinado módulo precisaremos re-compilar os outros.
- ▶ ALTERNATIVA: **Compilar os módulos separadamente e depois ligar os diversos módulos-objetos gerados para criar um executável.**

Compilando separadamente

```
> gcc -c converte.c  
> gcc -c principal.c  
> gcc -o exe converte.o principal.o
```

Conclusão

Conclusão

- ▶ Nesta aula, foi realizada uma revisão de **programação em C**.
- ▶ Foram apresentados recursos de programação e estruturação de dados fundamentais da linguagem.

Próxima Aula

- ▶ Esquema de memória.
- ▶ Alocação estática de memória vs Alocação dinâmica de memória.

Exercícios

Exercício 1

Implemente o jogo conhecido *pedra, papel, tesoura*. Neste jogo, o usuário e o computador escolhem entre *pedra*, *papel* ou *tesoura*. Sabendo que *pedra* ganha de *tesoura*, *papel* ganha de *pedra* e *tesoura* ganha de *papel*, exiba na tela o ganhador de cinco rodadas (melhor de cinco): usuário ou computador. Para essa implementação, assuma que o número 0 representa a *pedra*, 1 representa *papel* e 2 representa *tesoura*. Observação: seu algoritmo deve ser honesto: a escolha feita pelo computador não pode ser realizada baseada na escolha do usuário e vice-versa. Dica: reutilizar funções para gerar números aleatórios num intervalo pré-definido.

Exercício 2

Implemente uma função iterativa para calcular o máximo divisor comum de dois números inteiros positivos, $MDC(x, y)$. Utilize o algoritmo de *Euclides*. Esse algoritmo é baseado no fato de que se o resto da divisão x por y , representado por r , for igual a zero, y é o MDC. Se o resto r for diferente de zero, o MDC de x e y é igual ao MDC de y e r . O processo se repete até que o valor da divisão seja igual a zero. Sua função para cálculo do MDC deverá estar num módulo separado, que posteriormente será refatorado para incluir outras funções matemáticas de interesse.

Exercício 3

Reutilizando o tipo *Ponto*, previamente descrito, implemente um tipo estruturado para representar um círculo: ponto central (*Ponto*) e raio (*float*). Implemente também uma função para verificar se, dado um ponto, ele está ou não dentro de um círculo, conforme protótipo a seguir. Implemente a função *main()* para testar sua solução.

```
// a função retorna 1 se ponto pertencer ao círculo,  
// zero caso contrário  
int circ_interior (Circulo* c, Ponto* p);
```

Bibliografia

Bibliografia

Os conteúdos deste material, incluindo figuras, textos e códigos, foram extraídos ou adaptados do livro-texto indicado a seguir:



Celes, Waldemar and Cerqueira, Renato and Rangel, José

Introdução a Estruturas de Dados com Técnicas de Programação em C.
Elsevier Brasil, 2016. ISBN 978-85-352-8345-7.



Anany Levitin.

Introduction To Design And Analysis Of Algorithms.
3rd Edition, 2012.



T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein.

Introduction to Algorithms.

The MIT Press, 3rd edition, 2009. ISBN-13: 978-0-262-53305-8.